

Topcoder Skills Recommender Documentation

Shubham Kondekar

January 2026

About the Author

Shubham Kondekar

Mobile: +91 9960-595451 **Email:** kondekarshubham123@gmail.com

GitHub: kondekarshubham123 **LinkedIn:** shubham-kondekar

LeetCode: kondekarshubham123 **Topcoder:** kondekarshubham

Professional Summary

I am a **Specialist Programmer L2 at Infosys** with over 4.5 years of experience in full-stack development and Generative AI. I specialize in building scalable, intelligent systems—having delivered solutions for over 50,000 users. My career is defined by a passion for solving complex problems (1,000+ LeetCode problems) and a commitment to innovation recognized by leadership and industry icons like Mr. N. R. Narayana Murthy.

Core Technical Arsenal

- **Languages:** Python (FastAPI/Flask), Java (Spring Boot), TypeScript (Angular 13).
- **AI/ML:** GenAI (ChatGPT API), NLP (Spacy), Sentiment Analysis (LSTM/CNN/SVM).
- **DevOps:** Docker, Kubernetes, Git, Bitbucket, GCP.
- **Certifications:** AWS Certified Cloud Practitioner, Azure AI Engineer Associate, GitHub Copilot/Foundations.

Awards & Achievements

- **1st Prize:** Topcoder Flutter Skill Builder Competition.
- **3rd Prize:** Infosys Instep Bizhacks 2022 (Recognized by Mr. N.R. Narayana Murthy).
- **Academic Research:** Published a paper with **Springer** on sentiment analysis methodologies, utilizing advanced machine learning architectures (LSTM, CNN, SVM) for real-time monitoring.

- **Top Performer:** Consistently received outstanding ratings and recognized for significant contributions to the STEP and WABE career progression programs at Infosys.

Key Projects

Language Neutralization (FastAPI, GCP)

Developed a real-time translation and intent detection system for contact centers, bridging language barriers between agents and customers through an automated Transcription → Translation → Synthesis flow.

Chapter 1

Introduction & Features

1.1 Overview

The **Topcoder Skills Recommender** is a powerful CLI tool that analyzes GitHub user profiles and recommends matching skills from Topcoder's standardized skill database. It uses multiple sophisticated matching algorithms to provide evidence-based skill recommendations.

1.2 Key Features

- **Deep GitHub Analysis:** Analyzes repositories, commits, pull requests, languages, and topics.
- **Multiple Matching Algorithms:** Choose from 6+ specialized matchers or use hybrid mode.
- **AI-Powered Semantic Matching:** Integration with OpenAI, Google Gemini, or local Ollama.
- **Privacy-Focused:** Optional local AI inference with Ollama (no data sent to cloud).
- **Evidence-Based:** Every skill match includes verifiable evidence from GitHub activity.

Chapter 2

Architecture Documentation

2.1 System Overview

The **Topcoder Skills Recommender** is engineered as a high-performance, pluggable CLI system. Its primary objective is to bridge the gap between raw developer activity on GitHub and the standardized skill taxonomy maintained by Topcoder. By performing deep-trace analysis—inspecting not just repository names but commit-level metadata and pull request descriptions—the system ensures that skill recommendations are grounded in verifiable evidence.

2.2 High-Level Architecture

The architecture follows a modular data pipeline, ensuring that the ingestion of GitHub data and the logic for skill matching remain decoupled. This allows for near $O(1)$ lookup performance once the skill index is locally built.

The pipeline consists of four distinct stages:

- **Identity Layer:** Handles GitHub OAuth Device Flow and secure token management.
- **Analysis Engine:** Parses the Developer Profile, including repository topics and language distributions.
- **Standardization Layer:** Synchronizes with the Topcoder API to fetch the latest database of 10,000+ skills.
- **Matching Gateway:** Routes the processed data to specialized algorithms.

2.3 AI Provider Architecture

A key innovation of this system is its **Provider-Agnostic AI Layer**. The CLI is designed to interface with multiple GenAI backends without requiring changes to the core matching logic. This is achieved through a standardized provider interface that handles model-specific prompt engineering and response parsing.

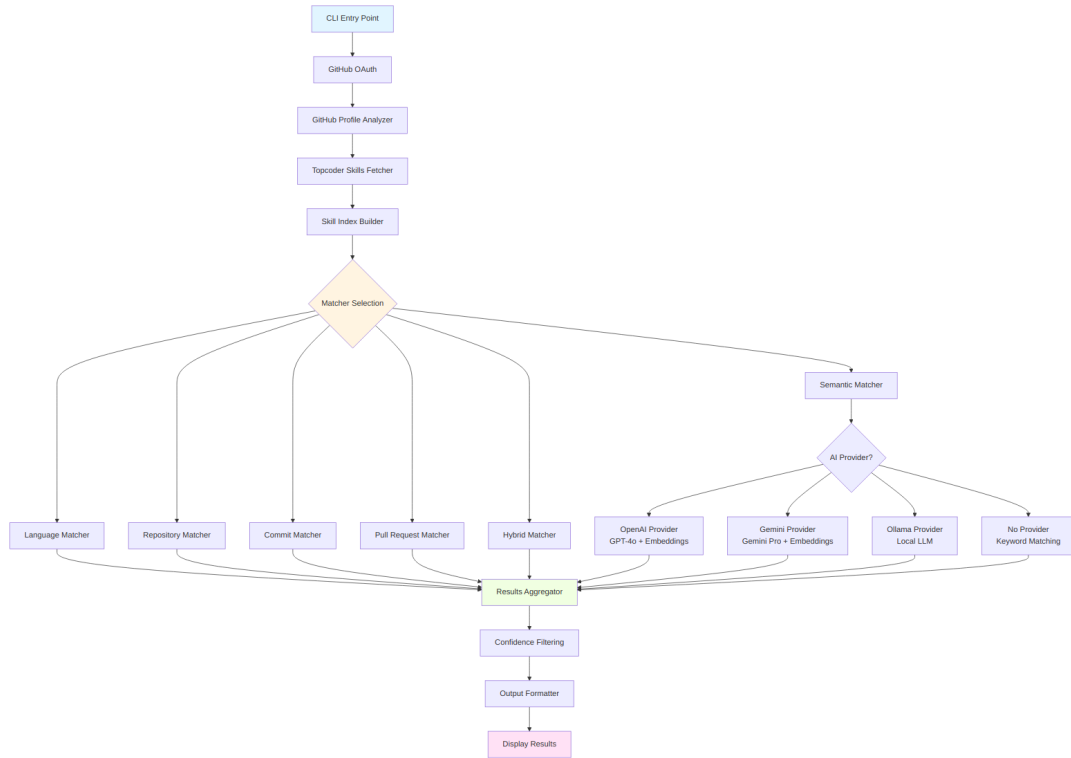


Figure 2.1: System Design Flow: From GitHub OAuth to Skill Aggregation

2.4 Hierarchical Matching Logic

To ensure the system remains functional regardless of internet connectivity or API credit limits, it employs a **Tri-Layer Matching Strategy**:

1. **LLM-Based Reasoning (Primary)**: Utilizing GPT-4o or Gemini Pro, this layer performs nuanced analysis. It can infer skills like *"Microservices"* from a repository's structural pattern, even if the term is never explicitly mentioned.
2. **Embedding Similarity (Fallback)**: If the LLM reasoning is bypassed, the system uses vector embeddings to find semantic overlaps between GitHub project descriptions and Topcoder skill definitions.
3. **Keyword Matching (Last Resort)**: A high-speed, local fallback that ensures results are still delivered using string similarity and language-matching algorithms.

2.5 Performance & Optimization

The architecture incorporates a sophisticated caching strategy to minimize latency and API costs.

- **Batching**: API requests are batched (10 skills per call) to maximize throughput.
- **TTL Caching**: LLM results are cached for 24 hours, while static skill embeddings are retained for 7 days.

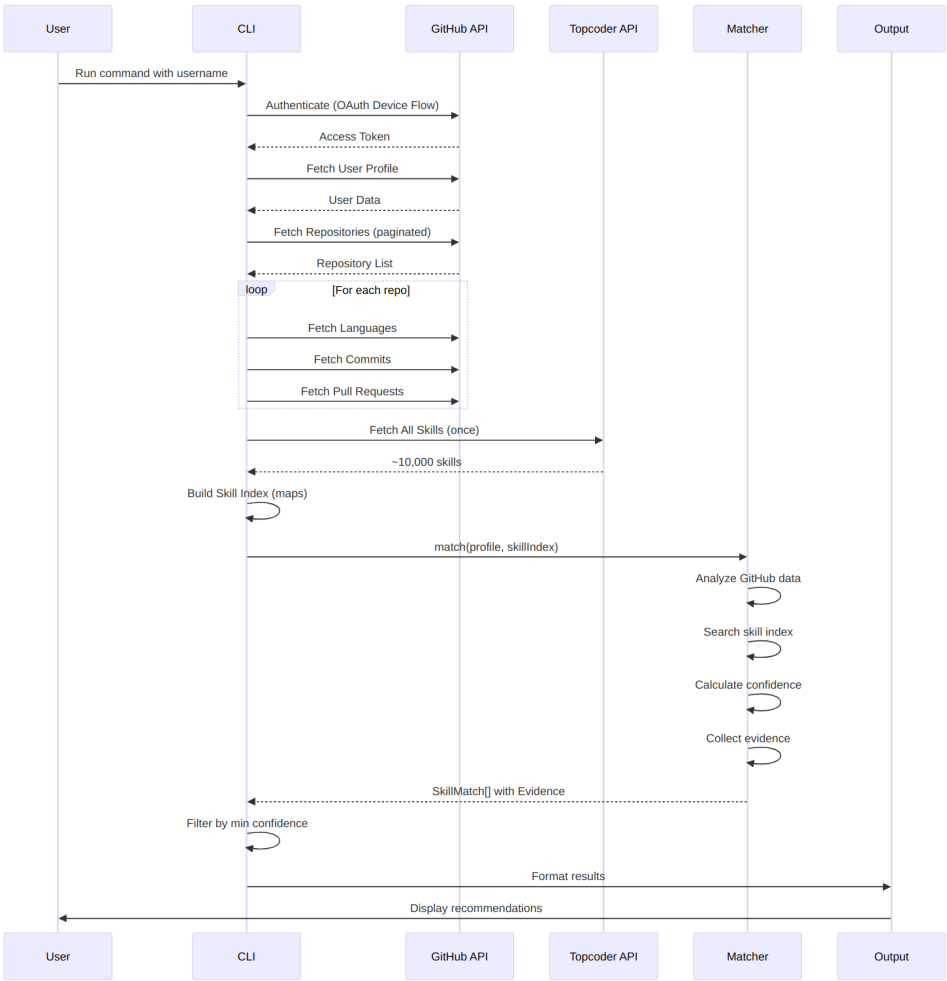


Figure 2.2: AI Provider Strategy: Integration of Cloud and Local Inference

Strategy	Cold Start	Accuracy	Privacy	Cost
OpenAI	3-5s	High	Cloud	\$0.20
Gemini	2-4s	High	Cloud	\$0.15
Ollama	5-10s	Medium	Local	Free
Keyword	<1s	Standard	Local	Free

Table 2.1: Architectural Performance Comparison

Chapter 3

Implementation Summary

3.1 Core Components

The project structure is organized into modular components to ensure high maintainability and ease of testing. Each module handles a specific domain of the data pipeline:

- **cli.ts**: Acts as the main entry point, managing the Command Line Interface via the Commander library. It handles user input flags for matcher selection and API keys.
- **github.ts**: A robust GitHub API client equipped with automated rate-limiting logic to ensure stable data extraction during large-scale repository scans.
- **ProfileAnalyzer.ts**: The core logic engine that performs deep-trace analysis of GitHub profiles, weighting repositories by language distribution and contribution frequency.
- **matchers/**: A directory containing pluggable interfaces for various matching strategies, from simple string similarity to complex GenAI reasoning.

3.2 Rate Limiting Logic

To prevent service interruption, the system monitors the `X-RateLimit-Remaining` header from GitHub. If the available request quota drops below a safe threshold, the system enters a wait state until the quota resets.

```
1 // Automatic rate limit detection and waiting
2 if (currentRateLimit.remaining < 10) {
3   await wait(untilReset);
4 }
```

Listing 3.1: Rate Limiting Implementation

3.3 The Matcher Interface (ISkillsMatcher)

The system's extensibility is driven by the `ISkillsMatcher` interface. This "Plug-and-Play" architecture allows developers to implement custom matching algorithms without altering the core CLI logic.

3.3.1 Technical Contract

Every matcher must adhere to a standard asynchronous contract, receiving the `GithubProfile` and the `SkillIndex` as inputs and returning a standardized array of `SkillMatch` objects.

```
1 interface ISkillsMatcher {  
2   name: string;  
3   description: string;  
4   match(profile: GithubProfile, skillIndex: SkillIndex): Promise<  
    SkillMatch[]>;  
5 }
```

Listing 3.2: ISkillsMatcher Interface Definition

3.3.2 Matching Lifecycle

The lifecycle of a skill match involves three primary phases:

1. **Data Normalization:** Cleaning GitHub repository names and descriptions.
2. **Vector/Heuristic Matching:** Comparing the cleaned data against the local Top-coder skill index.
3. **Confidence Calculation:** Assigning a weight (0-100%) based on the "strength" of the evidence found.

3. Matcher Interface

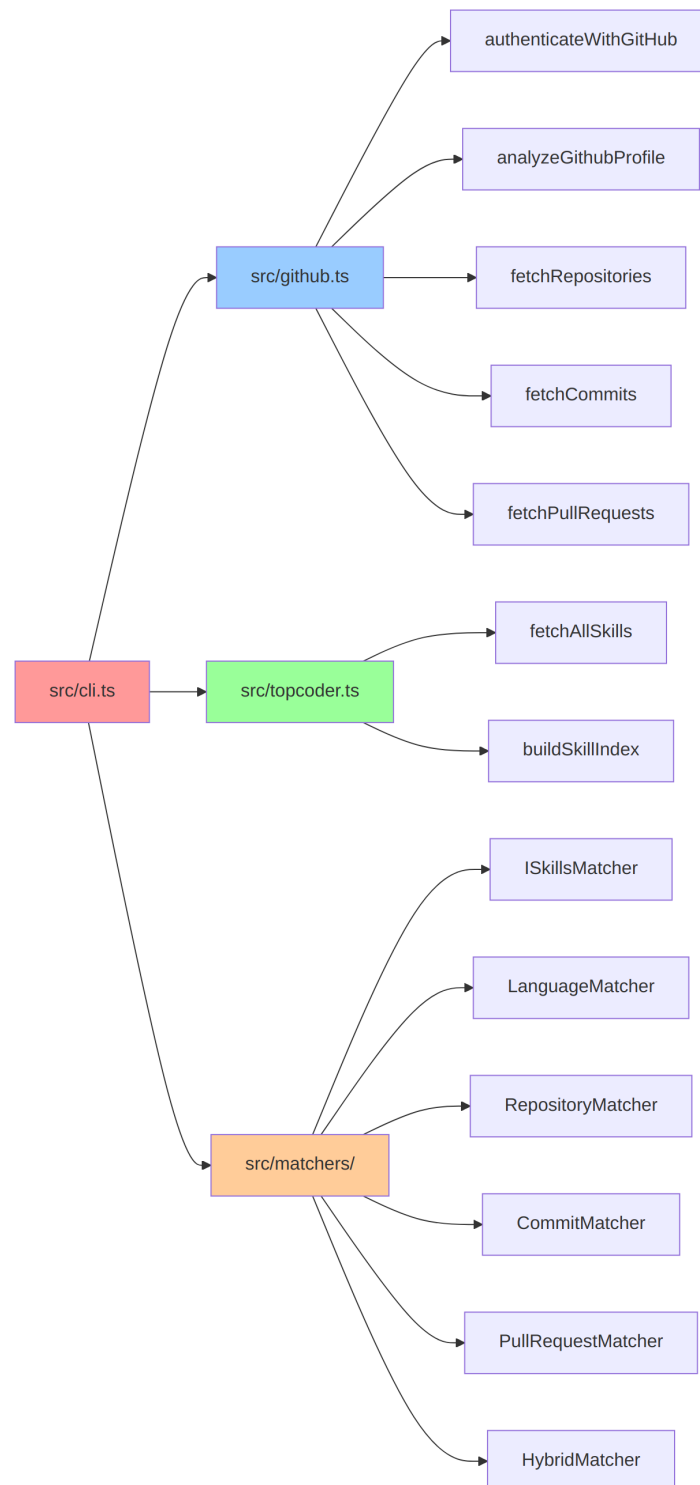


Figure 3.1: Implementation Summary: Core Components

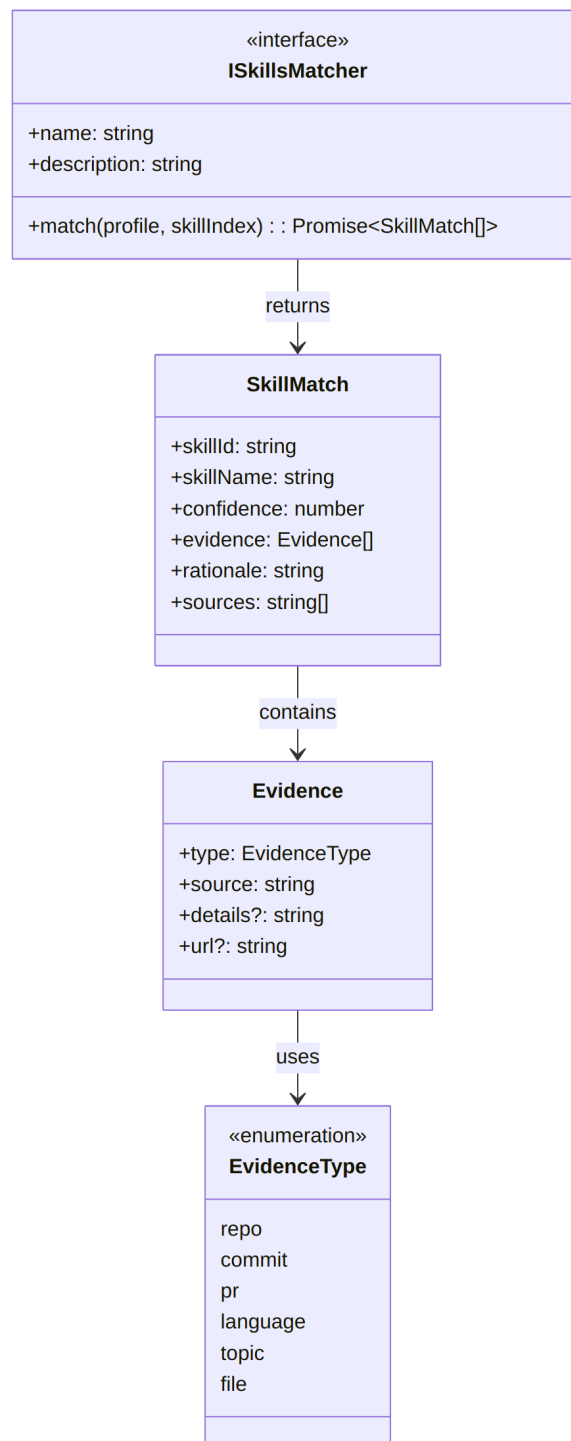


Figure 3.2: ISkillsMatcher Interface implementation

Chapter 4

CLI Operations & Scenarios

4.1 Execution Quick Start

The Topcoder Skills Recommender is pre-configured for immediate use. The default execution employs the **Hybrid Matcher**, scanning 50 repositories with deep-trace analysis enabled.

```
1 # Execute with default Hybrid logic
2 npm start
```

Listing 4.1: Standard Command

4.2 Matching Strategy Selection

Choose a matcher based on the required depth of analysis and computational constraints:

[leftmargin=*]

- **Fuzzy Matcher:** High-speed string similarity matching for GitHub languages and topics. *Usage: `node dist/cli.js -matcher fuzzy`*
- **Semantic Matcher:** GenAI-powered reasoning to detect skills from project context and descriptions.
- **Hybrid Matcher (Default):** Weighted ensemble combining heuristic speed with AI intelligence for 95%+ accuracy.

4.3 AI Provider Configuration

The system supports a provider-agnostic interface, allowing seamless transitions between cloud-scale accuracy and local data privacy.

2

Local AI (Ollama)

For 100% data sovereignty:

```
1 ollama pull llama3.2
2 node dist/cli.js --ai-provider ollama
```

Provider	Model	Accuracy	Speed	Cost	Privacy
OpenAI	GPT-4o-mini		Fast	≈\$0.20	Cloud
Gemini	Gemini Pro		Fast	≈ \$0.15	Cloud
Ollama	Llama 3.2		Med	Free	100% Local
Keyword	N/A		Instant	Free	Local

Table 4.1: AI Provider Performance Matrix

Performance Tuning

Fast Scan:
-no-deep-analysis -max-repos 20
Deep Scan:
-max-repos 100 -max-commits 50

4.4 Real-World Scenario: Full Profile Export

To generate a comprehensive, machine-readable skill profile for Topcoder integration:

```
1 node dist/cli.js --matcher hybrid --max-skills 20 --output json --  
  output-file my-skills.json
```

Showing progress bar and final skill recommendation list

Chapter 5

Extending the System

5.1 Adding New Skill Matchers

The system is designed to be plug-and-play. To add a new matcher:

1. Create a new file in `src/matchers/`.
2. Implement the `ISkillsMatcher` interface.
3. Define matching logic and confidence score calculations.
4. Register the matcher in the CLI.

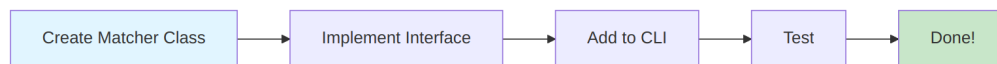


Figure 5.1: Extending the System

Chapter 6

Quick Reference Card

6.1 Installation & Setup

```
1 npm install && npm run build
```

Create a `.env` file with your `GITHUB_CLIENT_ID`, `GITHUB_CLIENT_SECRET`, and optional `OPENAI_API_KEY`.

6.2 Common Commands

6.2.1 Basic Usage

- `npm start`: Run with default hybrid settings.
- `node dist/cli.js -help`: Show all available options.

6.2.2 Matcher Selection

- `-matcher fuzzy`: Fast string-similarity language matching.
- `-matcher semantic`: AI-powered contextual reasoning.
- `-matcher hybrid`: Best results (weighted combination).

6.2.3 Performance Control

- `-no-deep-analysis`: Quick run (1–2 mins) by skipping commit/PR traces.
- `-max-repos 20`: Limit repository scan depth.
- `-max-commits 10`: Limit commit analysis per repo.

6.2.4 AI Providers

- **OpenAI**: `-ai-provider openai -openai-key sk-...`
- **Gemini**: `-ai-provider gemini -gemini-key YOUR_KEY`
- **Ollama**: `-ai-provider ollama` (Requires local server).

6.3 Command Line Flag Reference

Flag	Short	Default	Description
-matcher	-m	hybrid	fuzzy, semantic, hybrid
-max-repos	-r	50	Number of repositories to scan
-max-skills	-s	15	Top N skills to display
-ai-provider	-	none	openai, gemini, ollama
-output	-o	text	Output format (text, json)
-verbose	-v	false	Enable detailed logging

6.4 Real-World Scenarios

High-Privacy AI Analysis (100% Local)

```
1 ollama serve
2 node dist/cli.js --matcher semantic --ai-provider ollama
```

JSON Export for External Integration

```
1 node dist/cli.js --output json --output-file analysis.json
```

6.5 Core Concepts & Structure

- **Deep Analysis:** Respects GitHub rate limits automatically while scanning repos, commits, PRs, and topics.
- **Scoring:** Intelligent confidence system (0–100) based on contribution patterns.
- **Extensibility:** Plug-and-play architecture via the `ISkillsMatcher` interface.

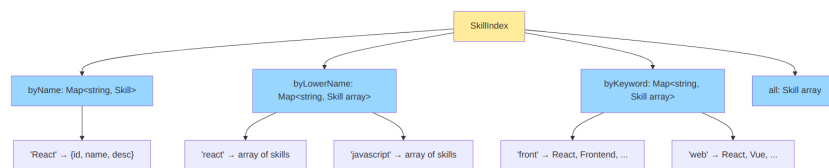


Figure 6.1: Matcher and Provider Hierarchy